HOMEWORK PROBLEM-2

QUES 1: Define an integer array. Declare an integer pointer and initialize it to point to first element of array. Use pointer arithmetic to traverse the array and print the elements. Modify the elements of the array using pointer arithmetic

CODE:

```
#include<stdio.h>

int main()

{

   int size;

   printf("enter the size of the array: ");

   scanf("%d",&size);

   int arr[size];

   printf("\nenter the elements: ");

   for(int i=0;i<size;i++)

   {

      scanf("%d",&arr[i]);

   }

   int *p=arr;


   for(int i=0;i<size;i++)

   {

      printf("%d ",*p);

      p++;

   }




   return 0;

}
```

QUES 2: Calculate string length using pointer

CODE:

```
#include <stdio.h>
int main()
{
   char str[100];
   printf("enter the string: ");
   scanf("%s",str);
   char *p;
   int length=0;
   p=str;
   while(*p!='\0')
   {
      length++;
      p++;
   }
   printf("the length of the string is: %d",length);
   return 0;
}
```

QUES 3: Write a function that calculate both the sum and product of the two numbers. Return these two results using pointer passed as arguments to the function.

CODE:

```
#include<stdio.h>
int sum(int *a,int *b)
{
   return *a+*b;
}
int product(int *a,int *b)
```

```
{
   return (*a)*(*b);
}
int main()
{
   int x,y;
   printf("enter two numbers: ");
   scanf("%d%d",&x,&y);
   printf("sum of the two numbers: %d\n",sum(&x,&y));
   printf("product of the two numbers: %d\n",product(&x,&y));
   return 0;
}
```

QUES 4: Create an array of structure uses dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access modify members

CODE:

```
#include <stdio.h>
#include<stdlib.h>

struct student
{
   int roll_no;
   char name[10];
};
int main()
{
   struct student *s;
   s = malloc(3 * sizeof(struct student));
```

```
    struct student *p=s;
    printf("enter the details of all the studentes: \n");
    for (int i = 0; i < 3; i++)
    {
        printf("enter roll no.");
        scanf("%d",&p->roll_no);
        printf("enter name: ");
        scanf("%s",p->name);
        p++;
    }
    printf("\nall details\n");
    p=s;
    for(int i=0;i<3;i++)
    {
        printf("roll no: %d\n",p->roll_no);
        printf("name: %s\n",p->name);
        p++;
    }
    free(s);
    return 0;
}
```

QUES 5: Define two functions with same signature, declare a function pointer and use the pointer to call both function.

CODE:

```
#include<stdio.h>
int sum(int *a, int *b)
```

```
{
   return *a+*b;
}
int product(int *a, int *b)
{
   return (*a)*(*b);
}
int main()
{
   int a=4,b=7;
   int(*p)(int*, int*);
   p=sum;
   printf("the sum: %d\n",p(&a,&b));
   p=product;
   printf("the product: %d",p(&a,&b));
   return 0;
}
```

QUES 6: Declare an integer a pointer to it and a double pointer. Print the values and addresses at each level

CODE:

```
#include<stdio.h>
int main()
{
   int a=3;
   int *x=&a;
   int **y=&x;
   printf("the value is: %d\n",a);
   printf("the address of the value is: %p\n",x);
```

```
    printf("the value using pointer is: %d\n",*x);
    printf("the value using double pointer is: %d\n",**y);
    return 0;
}
```

QUES 7: Sort an array of floating point numbers using selection sort.

CODE:

```
#include <stdio.h>
void swap(float *a, float *b)
{
    float temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
void selection(float arr[], int n)
{
    int min;
    for (int i = 0; i < n - 1; i++)
    {
        min = i;
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min])
            {
                min = j;
            }
        }
```

```
        swap(&arr[i], &arr[min]);
    }
}
int main()
{
    float arr[] = {4.5, 2.4, 7.9, 1.4, 9.8};
    int size = sizeof(arr) / sizeof(arr[0]);
    selection(arr, size);
    for (int i = 0; i < size; i++)
    {
        printf("%f ", arr[i]);
    }

    return 0;
}
```

QUES 8: Define a structure student with id , name and grade create an array of student structures and sort them based on different feed

CODE:

```
#include <stdio.h>
struct student
{
    int id;
    char name[25];
    char grade;
};
int main()
{
```

```
int size;
printf("enter no. of students: ");
scanf("%d",&size);
struct student s[size];
for(int i=0;i<size;i++)
{
   printf("enter id of student %d: ",i+1);
   scanf("%d",&s[i].id);
   printf("enter name of student: %d: ",i+1);
   scanf("%s",s[i].name);
   printf("enter grade of student %d: ",i+1);
   scanf(" %c",&s[i].grade);
}
printf("\nthe sorted details of the students are: \n");
printf("all ids: \n");
for(int i=0;i<size;i++)
{
   printf("%d\n",s[i].id);
}
printf("all names: \n");
for(int i=0;i<size;i++)
{
   printf("%s\n",s[i].name);
}
printf("all grades: \n");
for(int i=0;i<size;i++)
{
   printf("%c\n",s[i].grade);
}
```

```
    return 0;
}
```

QUES 9: Sort a set of negative numbers using radix sort

CODE:

```
#include <stdio.h>
void countSort(int arr[], int n, int place)
{
    int output[n];
    int count[10] = {0};

    for (int i = 0; i < n; i++)
    {
        int digit = (arr[i] / place) % 10;
        count[digit]++;
    }
 for (int i = 1; i < 10; i++)
    {
        count[i] = count[i] + count[i - 1];
    }
 for (int i = n - 1; i >= 0; i--)
    {
        int digit = (arr[i] / place) % 10;

        output[count[digit] - 1] = arr[i];
        count[digit]--;
    }
for (int i = 0; i < n; i++)
```

```
    {
        arr[i] = output[i];
    }
}
void radixSort(int arr[], int n)
{
    int max = arr[0];

    for (int i = 1; i < n; i++)
    {
        if (arr[i] > max)
            max = arr[i];
    }
 for (int place = 1; max / place > 0; place *= 10)
    {
        countSort(arr, n, place);
    }
}
int main()
{
    int n;
printf("Enter number of elements: ");
    scanf("%d", &n);
 int arr[n];
printf("Enter elements:\n");
 for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
```

```
 int positive[n], negative[n];
int p = 0, neg = 0;
   for (int i = 0; i < n; i++)
   {
      if (arr[i] < 0)
      {
         negative[neg] = -arr[i];
         neg++;
      }
      else
      {
         positive[p] = arr[i];
         p++;
      }
   }
 if (neg > 0)
      radixSort(negative, neg);
if (p > 0)
      radixSort(positive, p);
 printf("Sorted array:\n");
for (int i = neg - 1; i >= 0; i--)
   {
      printf("%d ", -negative[i]);
   }
 for (int i = 0; i < p; i++)
   {
      printf("%d ", positive[i]);
   }
 return 0;
```

```
}
```

QUES 10: Write a C program to copy one array to another using pointers

CODE:

```
#include <stdio.h>

int main()
{
   int n;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   int arr1[n], arr2[n];

   int *p = arr1;
   int *q = arr2;

   printf("Enter elements:\n");

   for (int i = 0; i < n; i++)
   {
      scanf("%d", p);
      p++;
   }

   p = arr1;

   for (int i = 0; i < n; i++)
```

```
    {
        *q = *p;
        p++;
        q++;
    }

    printf("Copied array:\n");

    q = arr2;

    for (int i = 0; i < n; i++)
    {
        printf("%d ", *q);
        q++;
    }

    return 0;
}
```